Linux Fundamentals

VNTECHIES

Today's Lesson

What is Linux & Linux distribution

Ubuntu server

Command line interface

File System Hierarchy & Navigation

Process Management

Package Management

What is Linux & Linux distribution

- **Linux** is an open-source operating system based on Unix. It acts as a bridge between hardware and software, managing system resources and enabling user interaction.
- **Linux Distributions** often shortened to "distro," is a packaged version of Linux that comes with the Linux kernel plus a collection of software and utilities that make the OS functional and user-friendly.

Distribution	Base / Family	Package Manager	Key Features	Common Use Cases
Ubuntu	Debian	apt	Beginner-friendly, large community, frequent releases	Desktop, servers, cloud
Debian	Independent	apt	Very stable, conservative updates	Servers, enterprise
Fedora	Independent (upstream for RHEL)	dnf	Cutting-edge, upstream for Red Hat	Developers, testing new tech
CentOS Stream	RHEL	dnf	Rolling-release between Fedora & RHEL	Servers, enterprise (free RHEL alternative)
Red Hat Enterprise Linux (RHEL)	Fedora	dnf	Commercial support, enterprise-ready	Corporations, datacenters
openSUSE	Independent	zypper	Two flavors: Leap (stable), Tumbleweed (rolling)	Developers, servers
Arch Linux	Independent	pacman	DIY, rolling release, minimal by default	Advanced users, customization
…				

Why Linux and Not something else

- Linux powers 78.3% of web-facing servers in 2025, maintaining a strong lead in hosting environments.
- Government data centers worldwide now report 64.9% Linux adoption, citing lower cost and better control.
- Banking and financial services increased their Linux infrastructure footprint by 21.5% YoY.
- Among Fortune 500 companies, 72.6% run mission-critical workloads on Linux.
- Linux is the OS of choice for high-frequency trading systems due to its customization and latency advantages.
- … And it go on and on

Why Ubuntu Server and not the other?

Ubuntu leads all Linux distributions with a **33.9%** market share, making it the most widely used Linux OS.

Debian ranks second with 16%, known for its stability and long-term support.

CentOS holds 9.3%, favored in server environments before its recent shift to CentOS Stream.

Other notable distributions include Red Hat (0.8%), Gentoo (0.5%), and Fedora (0.2%).

Ubuntu Server

Ubuntu can be installed on:

- Physical Machine (Bare Metal)
- Virtual Machine (VM)
- Provided by Cloud Provider (AWS, GCP, Azure, …)
- Containers (Lightweight Option)

Command Line Interface (CLI)

A **Command Line Interface (CLI)** is a text-based way of interacting with a computer system, unlike a **Graphical User Interface (GUI)** that uses windows, icons, and menus. In a CLI, you type commands into a terminal (or console), and the system executes them, often returning text-based output.

Command Line Interface (CLI)

Terminal / Console

The environment where you enter commands.

Example: **GNOME Terminal** (Linux), **Command Prompt** or **PowerShell** (Windows).

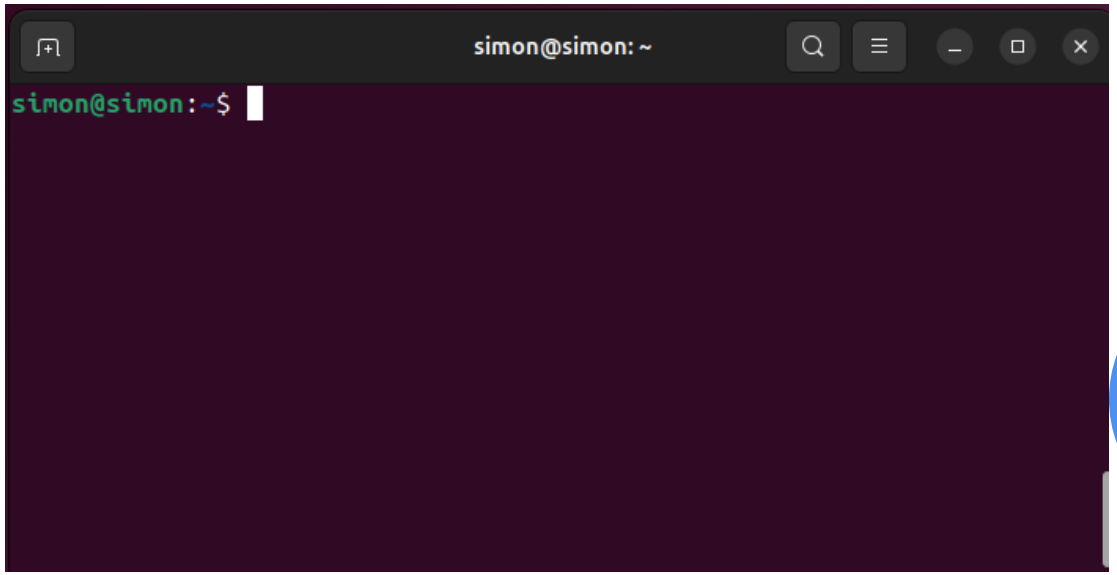
Shell

The program that interprets the commands you type and communicates with the operating system.

Examples: **Bash** (Linux/Unix, macOS), **Zsh** (Linux/macOS), **PowerShell** (Windows)

Prompt

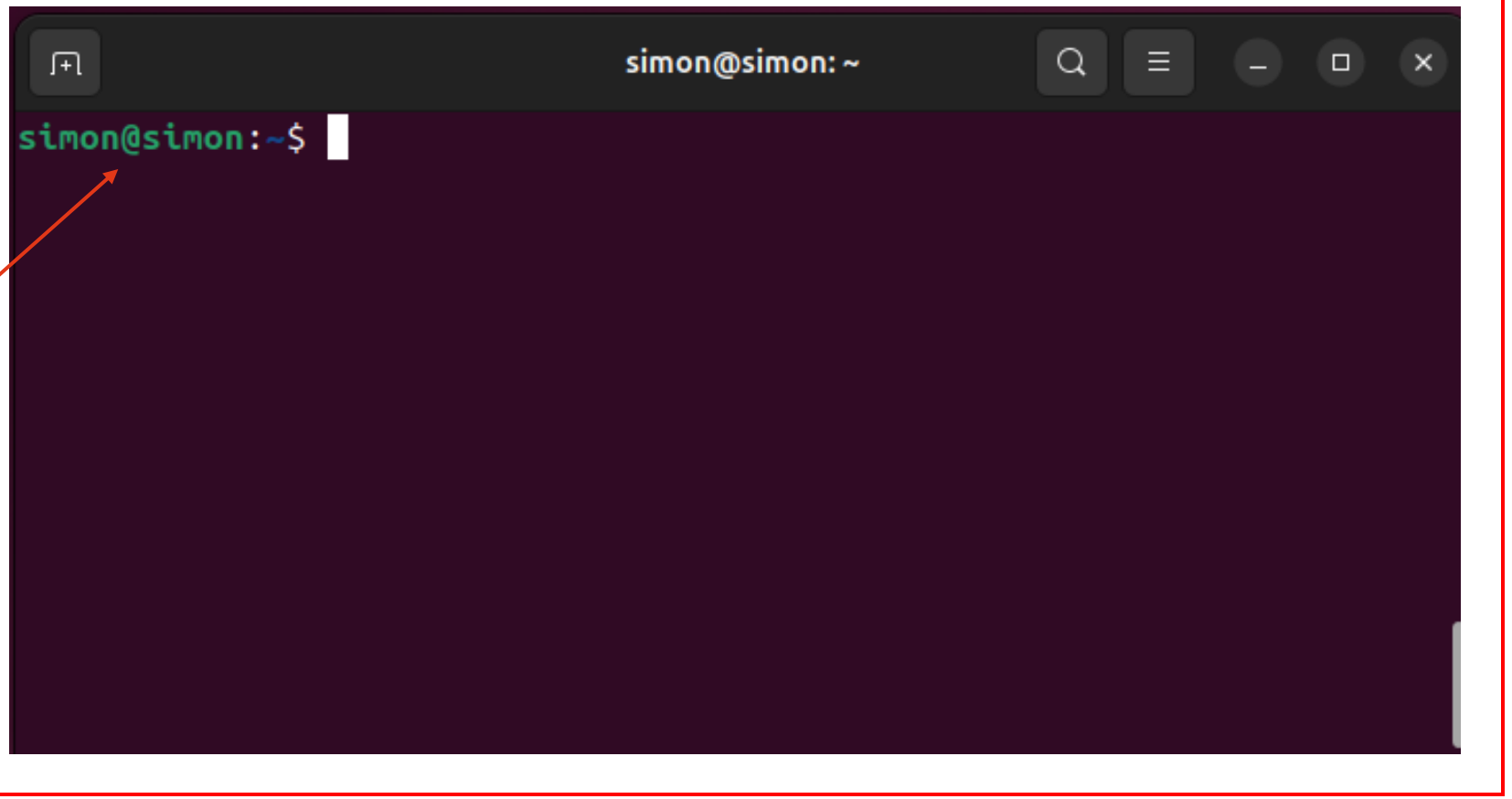
A symbol or text displayed by the shell indicating it's ready for input.



VNTECHIES

Command Line Interface (CLI)

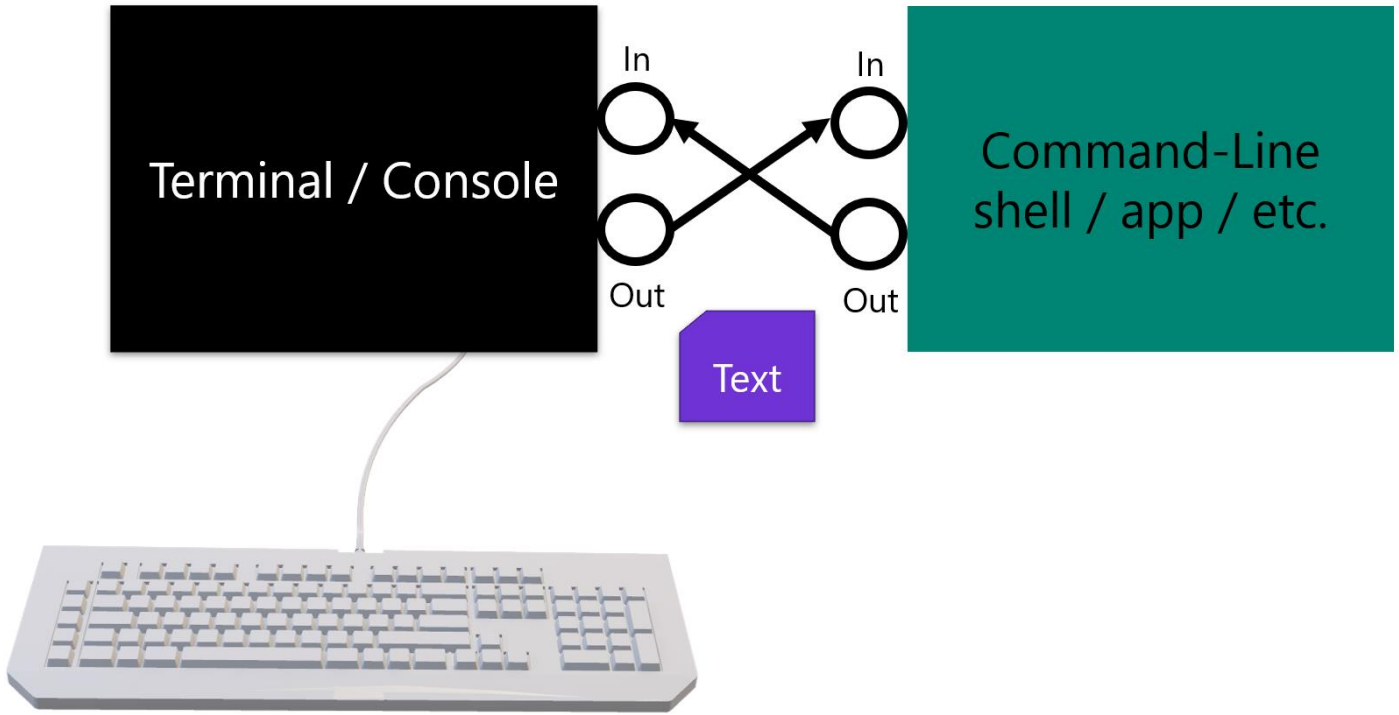
Terminal / Console

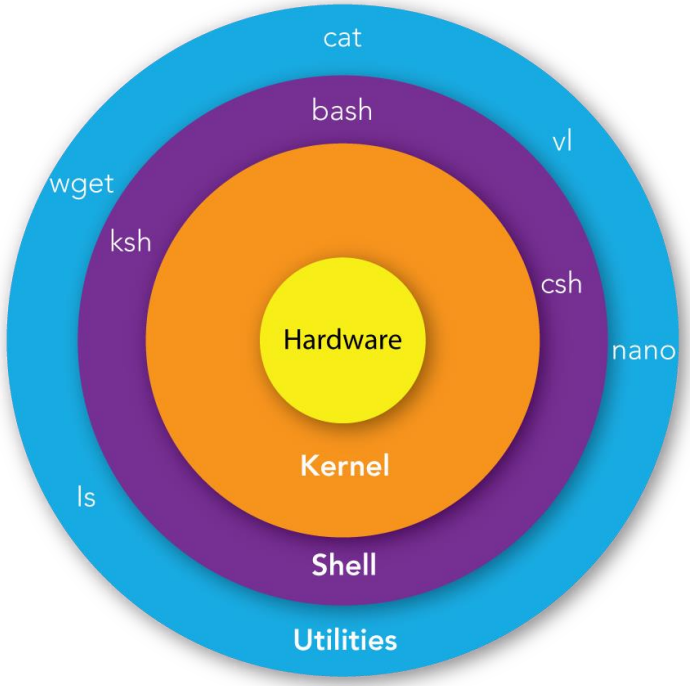


Prompt

Shell?

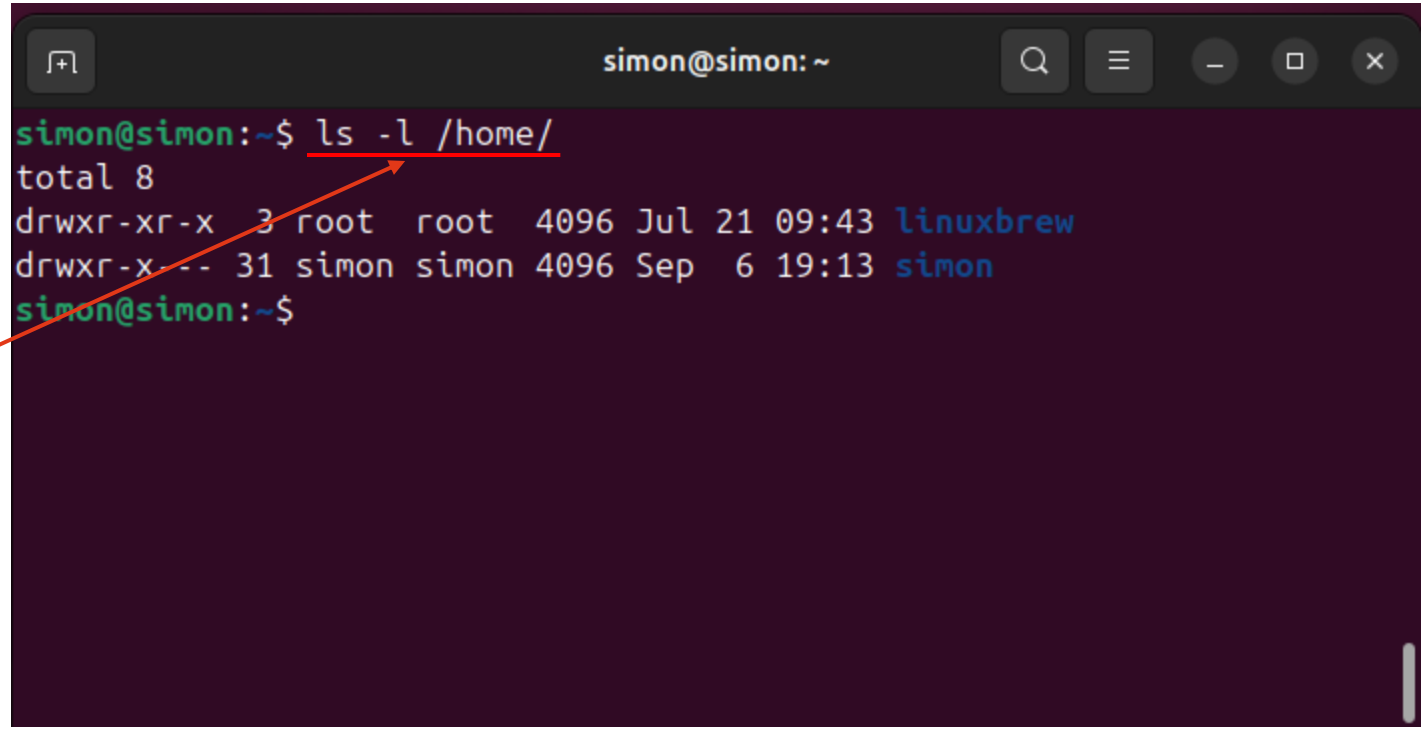
Shell


Terminal / Console
In
Out
In
Out
Command-Line
shell / app / etc.
Text


cat
bash
vl
wget
ksh
csh
Hardware
nano
Kernel
ls
Shell
Utilities

VNTECHIES

Command



VNTECHIES

Command

Command

Instructions (executable binary, script) you type into the CLI.

Examples: `ls -l /home/`

Arguments:

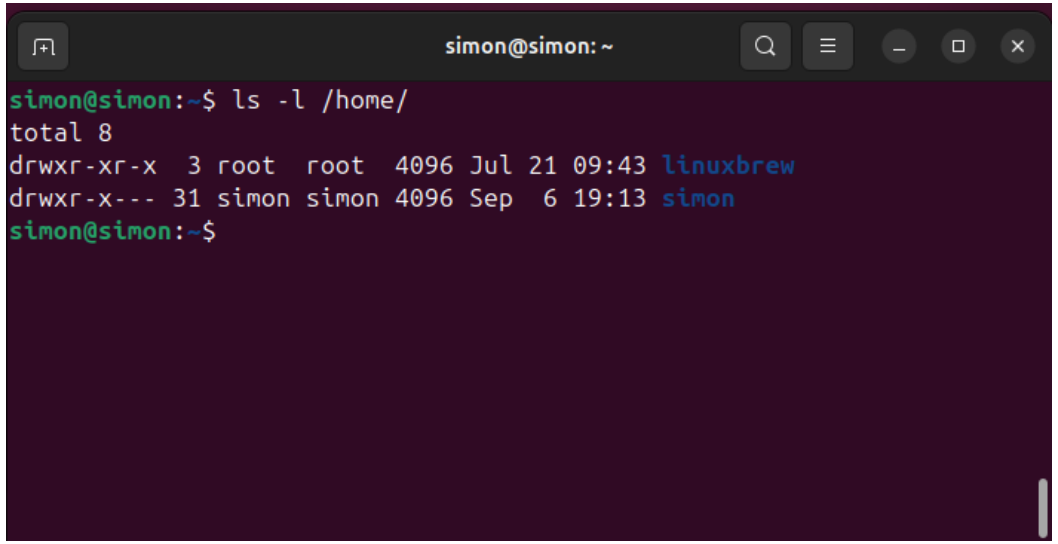
Extra information you pass to a command

Examples: `/home/`

Options / Flags:

Modify the behavior of a command, usually starting with `-` or `--`.

Examples: `-l`

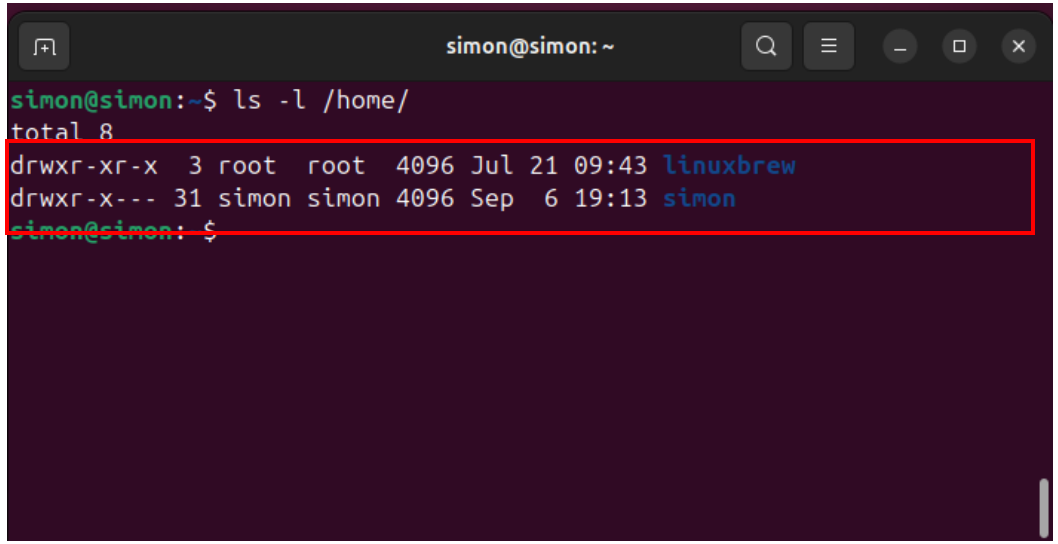


Command

Output

The result the command returns, shown as text in the terminal.

Example: ls outputs a list of files.

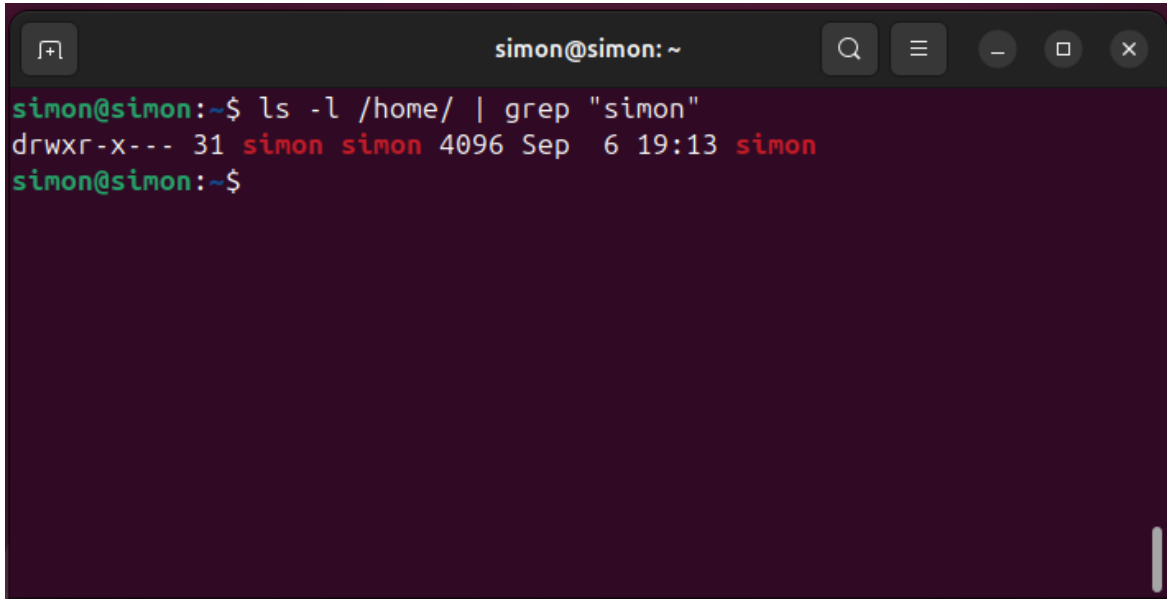


Command

Pipelines

Using (|) to send output of one command as input to another.

Example: `ls | grep “simon”`

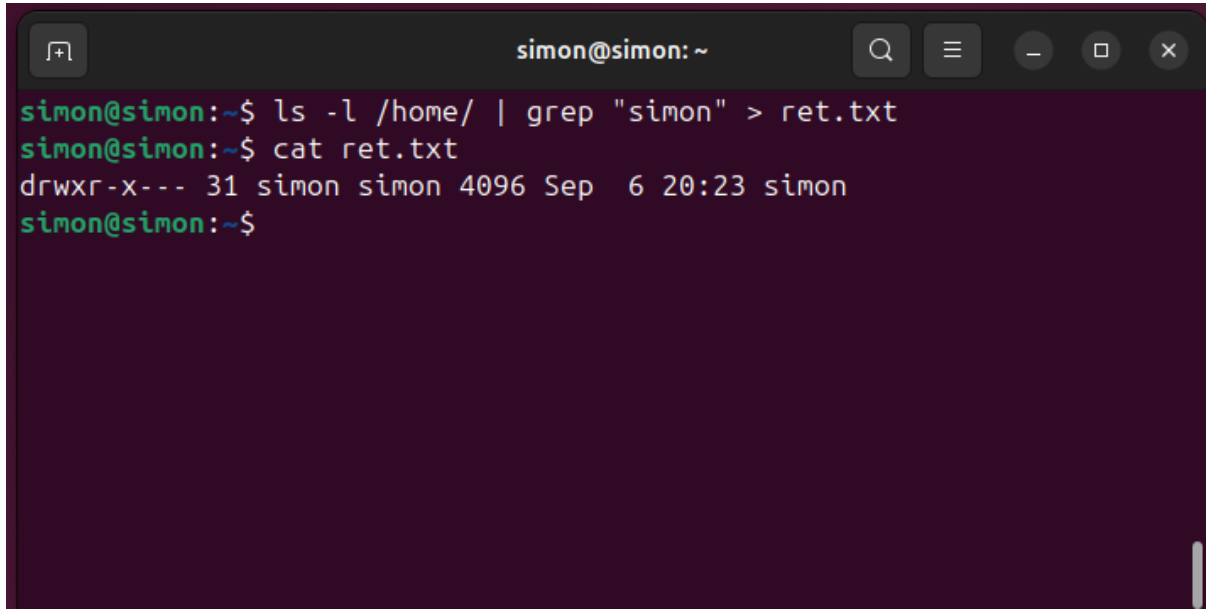


Command

Redirection

(`>`, `>>`, `<`): Save or read command output/input from a file.

Example: `ls > files.txt`

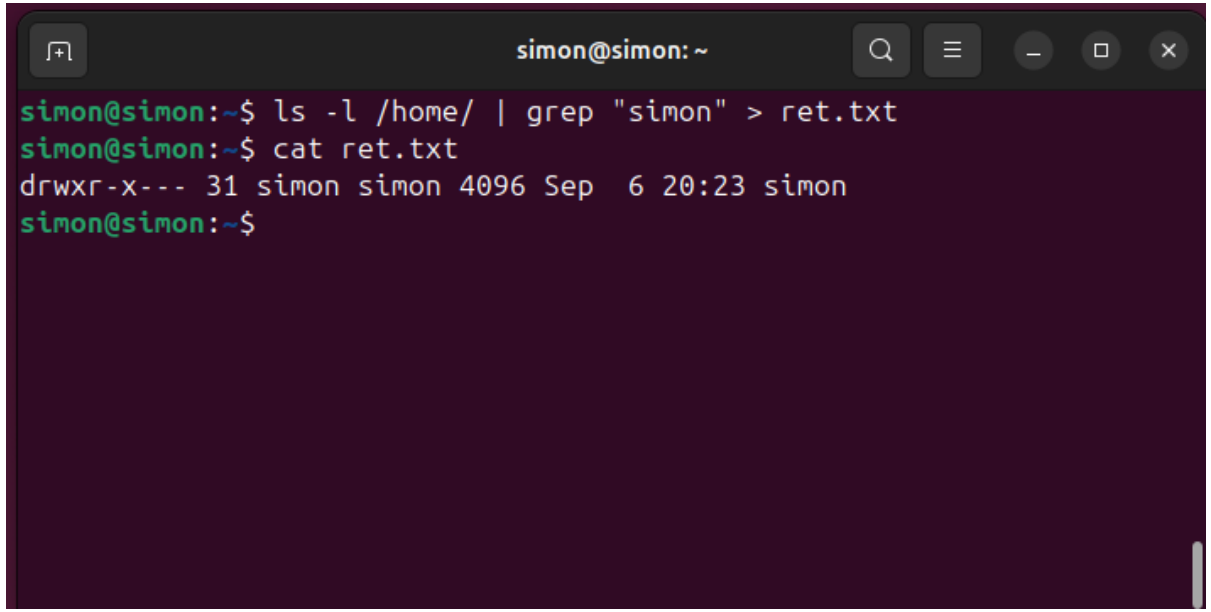


Command

Redirection

(`>`, `>>`, `<`): Save or read command output/input from a file.

Example: `ls > files.txt`



STDOUT, STDERR, Exit Code

STDOUT (Standard Output)

The default stream where a program writes its normal output (results).

By default, it goes to the terminal screen.

STDERR (Standard Error)

The default stream where a program writes error messages.

By default, it also appears on the terminal, but it's separate from stdout.

Exit Code (Return Code / Exit Status)

A small number (0–255) returned by a process when it finishes.

It tells whether the command succeeded or failed:

- 0 → success
- Non-zero → error (different numbers mean different error types)

STDOUT, STDERR, Exit Code

Redirection Examples

Redirect **stdout** to a file:

```
ls > files.txt
```

Redirect **stderr** to a file:

```
ls /not_exist 2> errors.txt
```

Redirect **both stdout + stderr**:

```
ls /not_exist > all.txt 2>&1
```

A file descriptor is just a number that the operating system uses to keep track of open files, devices, or data streams.

0 → stdin (standard input)

1 → stdout (standard output)

2 → stderr (standard error)

So when you see 1 or 2 in redirection, they refer to those streams.

- &1 Meaning

& tells the shell "this is a file descriptor, not a file name".

1 is the file descriptor for stdout.

So &1 means: “use the same place stdout (1) is going to.”

STDOUT, STDERR, Exit Code

Exit Code (Return Code / Exit Status)

A small number (0–255) returned by a process when it finishes.

It tells whether the command succeeded or failed:

- 0 → success
- Non-zero → error (different numbers mean different error types)

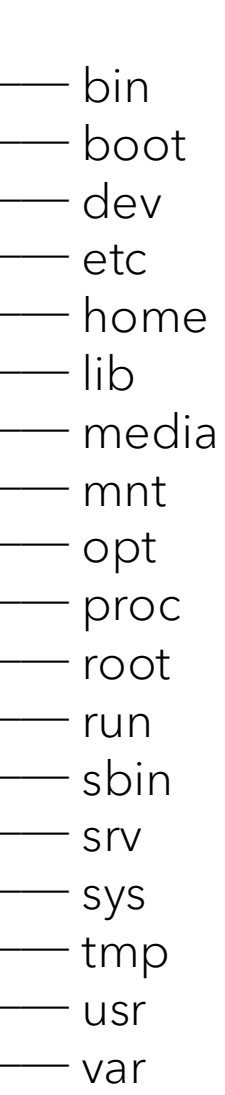
Examples:

```
ls
echo $?   # should print 0 (success)
ls /not_exist
echo $?   # should print non-zero (e.g., 2)
```

File System Overview

File system

Ubuntu, like all Linux distributions, follows the **Filesystem Hierarchy Standard (FHS)**.
This means everything is organized in a **single tree structure** starting from the **root directory /**



File system

Directory	Purpose
/	Root directory – the top of the file system hierarchy. Everything starts here.
/bin	Essential **user binaries** (programs) like ls, cp, mv, cat. Needed for basic system operation.
/boot	Boot loader files (like **GRUB**), Linux kernel (vmlinuz), initrd images. Required to boot the system.
/dev	Device files (represent hardware like disks /dev/sda, terminals /dev/tty, USB /dev/usb*).
/etc	System-wide **configuration files** (like /etc/passwd, /etc/hosts, /etc/ssh/sshd_config).
/home	User home directories (e.g., /home/simon), personal files, and settings.
/lib, /lib64	Essential **shared libraries** and kernel modules required by programs in /bin and /sbin.
/media	Mount points for removable media (USB drives, CDs, etc.).
/mnt	Temporary mount point (for manually mounted filesystems, e.g., extra hard disks).
/opt	Optional software, usually third-party apps not part of the core system.
/proc	Virtual filesystem providing info about running processes and the kernel (e.g., /proc/cpuinfo).
/root	Home directory for the **root user** (administrator).
/run	Stores temporary runtime data for processes (since boot).
/sbin	System binaries – administrative commands (fdisk, iptables, ifconfig).
/srv	Data for services (like web servers /srv/www).
/sys	Virtual filesystem providing info about devices and drivers.
/tmp	Temporary files (deleted at reboot).
/usr	Secondary hierarchy for **user applications and files**. Contains:

Navigation command

cd (change directory) → Moves between directories in the filesystem.
Example: cd /home/user/Documents

ls (list) → Displays the contents of a directory.
Example: ls -l (long listing with details)

pwd (print working directory) → Shows the current directory path.
Example: pwd → /home/user/Documents

find → Searches for files and directories based on criteria like name, size, or permissions.
Example: find /home -name "*.txt"

locate → Quickly searches files using a pre-built database (faster than find).
Example: locate notes.txt

which → Shows the path of an executable command (first one found in your $PATH).
Example: which python3 → /usr/bin/python3

whereis → Locates the binary, source, and man page for a command.
Example: whereis gcc → /usr/bin/gcc /usr/share/man/man1/gcc.1.gz

Absolute vs relative paths

Absolute Path:
The complete path from the root (/) directory to a file or folder.

Always starts with /.

Independent of the current working directory.

Example: /home/user/Documents/file.txt

Relative Path:
The path to a file or directory relative to your current location.

Does not start with /.

Depends on where you are (pwd).

Examples:

./file.txt → file in the current directory

../notes.txt → file in the parent directory

Absolute vs relative paths

Absolute Path:
The complete path from the root (/) directory to a file or folder.

Always starts with /.

Independent of the current working directory.

Example: /home/user/Documents/file.txt

Relative Path:
The path to a file or directory relative to your current location.

Does not start with /.

Depends on where you are (pwd).

Examples:

./file.txt → file in the current directory

../notes.txt → file in the parent directory

File, Directory Permission

Every file and directory in Linux has **three types of permissions** for **three categories of users**:

User Categories:

- **User (u)** – The owner of the file
- **Group (g)** – Users in the same group as the file
- **Others (o)** – Everyone else

Permission Types

Symbol	Meaning	Description
r	Read	View contents of a file or list a directory
w	Write	Modify contents of a file or directory
x	Execute	Run the file as a program or access a directory

File, Directory Permission

Breakdown:

- → Regular file (could be d for directory)

rwx → User (dao) has read, write, execute

r-x → Group (devs) has read and execute

r-- → Others have read only

Number	Permissions	Binary	Description
0		000	No permissions
1	--x	001	Execute only
2	-w-	010	Write only
3	-wx	011	Write and execute
4	r--	100	Read only
5	r-x	101	Read and execute
6	rw-	110	Read and write
7	rwx	111	Read, write, and execute

Changing Permissions

Use the chmod command:

Symbolic mode:

- chmod u+x file → Add execute for user
- chmod g-w file → Remove write for group

Numeric mode:

- chmod 755 file → Sets rwxr-xr-x
  - 7 = rwx (user)
  - 5 = r-x (group)
  - 5 = r-x (others)

Number	Permissions	Binary	Description
0		000	No permissions
1	--x	001	Execute only
2	-w-	010	Write only
3	-wx	011	Write and execute
4	r--	100	Read only
5	r-x	101	Read and execute
6	rw-	110	Read and write
7	rwx	111	Read, write, and execute

Changing Ownership

- chown dao file → Change owner to dao
- chgrp devs file → Change group to devs

Linux Process Management

What is a Process?

A process is an instance of a running program. When you execute a command or launch an application, the operating system creates a process to manage its execution. Each process has its own memory space, system resources, and execution context.

Processes are fundamental to multitasking in Linux, allowing multiple programs to run simultaneously. They can be interactive (like a terminal session) or background services (like a web server).

```
simon@simon ~
-: ps aux | head -n 10
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1  23720 14412 ?        Ss   13:07   0:02 /sbin/init splash
root           2  0.0  0.0      0     0 ?        S    13:07   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    13:07   0:00 [pool_workqueue_release]
root           4  0.0  0.0      0     0 ?        I<   13:07   0:00 [kworker/R-rcu_gp]
root           5  0.0  0.0      0     0 ?        I<   13:07   0:00 [kworker/R-sync_wq]
root           6  0.0  0.0      0     0 ?        I<   13:07   0:00 [kworker/R-kvfree_rcu_reclaim]
root           7  0.0  0.0      0     0 ?        I<   13:07   0:00 [kworker/R-slub_flushwq]
root           8  0.0  0.0      0     0 ?        I<   13:07   0:00 [kworker/R-netns]
root          11  0.0  0.0      0     0 ?        I<   13:07   0:00 [kworker/0:0H-events_highpri]
```

Process State

Every running program in Linux is a **process**, and each has unique identifiers and states.

- PID (Process ID): Unique number assigned to each process.
- PPID (Parent Process ID): The PID of the process that started it.
- Process States:

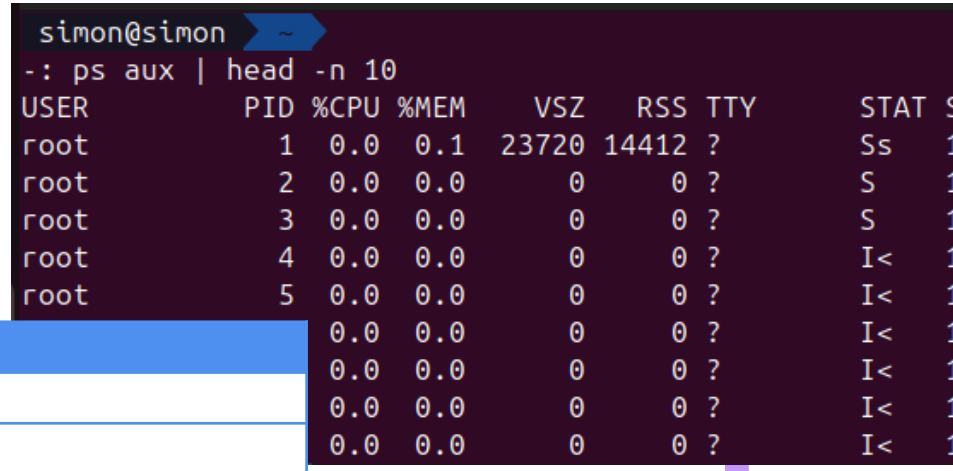
```
simon@simon ~
-: ps aux | head -n 10
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT S
root           1  0.0  0.1  23720 14412 ?        Ss   1
root           2  0.0  0.0      0     0 ?        S    1
root           3  0.0  0.0      0     0 ?        S    1
root           4  0.0  0.0      0     0 ?        I<   1
root           5  0.0  0.0      0     0 ?        I<   1
               0.0  0.0      0     0 ?        I<   1
               0.0  0.0      0     0 ?        I<   1
               0.0  0.0      0     0 ?        I<   1
               0.0  0.0      0     0 ?        I<   1
```

Code	State Name	Description
R	Running	Actively executing or ready to run.
S	Sleeping	Waiting for an event (e.g., I/O). Interruptible.
D	Uninterruptible Sleep	Waiting for hardware or system resource. Cannot be interrupted.
Z	Zombie	Process has terminated but still has an entry in the process table.
T	Stopped	Suspended by a signal or job control.
t	Tracing Stop	Being traced or debugged.
X	Dead	Should never be seen–used internally.
K	Wakekill	Woken up to be killed. Rare.
W	Paging	Process is paging memory (obsolete in modern kernels).
I	Idle	Kernel thread is idle (seen in newer kernels).

Process State

Every running program in Linux is a **process**, and each has unique identifiers and states.

- PID (Process ID): Unique number assigned to each process.
- PPID (Parent Process ID): The PID of the process that started it.
- Process States:



Code	State Name	Description
R	Running	Actively executing or ready to run.
S	Sleeping	Waiting for an event (e.g., I/O). Interruptible.
D	Uninterruptible Sleep	Waiting for hardware or system resource. Cannot be interrupted.
Z	Zombie	Process has terminated but still has an entry in the process table.
T	Stopped	Suspended by a signal or job control.
t	Tracing Stop	Being traced or debugged.
X	Dead	Should never be seen–used internally.
K	Wakekill	Woken up to be killed. Rare.
I	Idle	Kernel thread is idle (seen in newer kernels).

Process Monitoring

Command	Description
ps	Snapshot of current processes
top	Real-time system resource usage
htop	Enhanced version of top (interactive)
pstree	Tree view of processes

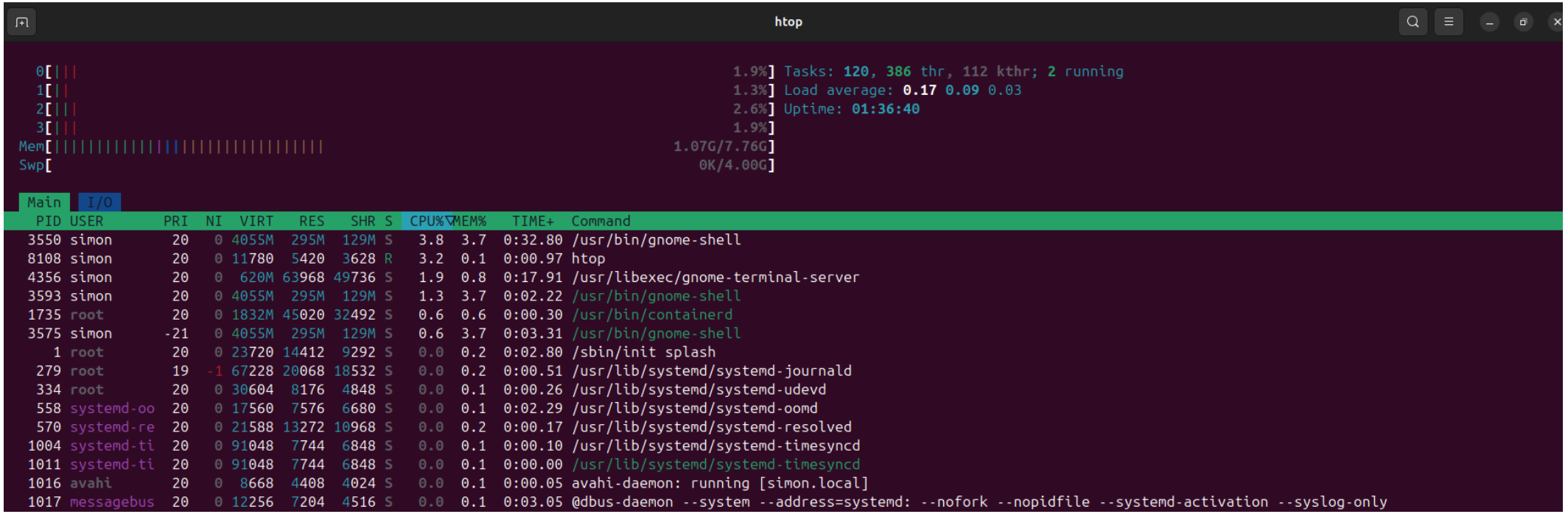
Process Monitoring

ps - report a snapshot of the current processes.

```
simon@simon ~
-: ps -ef | head -n 10
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 13:07 ?        00:00:02 /sbin/init splash
root           2       0  0 13:07 ?        00:00:00 [kthreadd]
root           3       2  0 13:07 ?        00:00:00 [pool_workqueue_release]
root           4       2  0 13:07 ?        00:00:00 [kworker/R-rcu_gp]
root           5       2  0 13:07 ?        00:00:00 [kworker/R-sync_wq]
root           6       2  0 13:07 ?        00:00:00 [kworker/R-kvfree_rcu_reclaim]
root           7       2  0 13:07 ?        00:00:00 [kworker/R-slub_flushwq]
root           8       2  0 13:07 ?        00:00:00 [kworker/R-netns]
root          11       2  0 13:07 ?        00:00:00 [kworker/0:0H-events_highpri]
```

Process Monitoring

top | htop - dynamic real-time view of a running system. It can display system summary information as well as a list of processes or threads currently being managed by the Linux kernel.



Process Monitoring

pstree - display a tree of processes

```
simon@simon ~
-: pstree -lp | head -n 10
systemd(1)-+-ModemManager(1159)-+-{ModemManager}(1166)
           |                    |-{ModemManager}(1167)
           |                    `-{ModemManager}(1169)
           |-NetworkManager(1080)-+-{NetworkManager}(1111)
           |                      |-{NetworkManager}(1112)
           |                      `-{NetworkManager}(1113)
           |-VBoxDRMClient(2765)-+-{VBoxDRMClient}(2776)
           |                     |-{VBoxDRMClient}(2777)
           |                     |-{VBoxDRMClient}(2779)
           |                     |-{VBoxDRMClient}(2780)
```

Process Control

Control and manage processes using these commands

Command	Function
kill PID	Send signal to terminate a process by process ID
killall	sends a signal to all processes running any of the specified commands.

Process Control

kill - Send signal to terminate a process by process ID

```
simon@simon ~
-: sleep 200 &
[1] 8981
simon@simon ~
-: ps -ef | grep sleep
simon       8981    4363  0 14:58 pts/0    00:00:00 sleep 200
simon       8994    4363  0 14:58 pts/0    00:00:00 grep --color=aut
-exclude-dir=.tox --exclude-dir=.venv --exclude-dir=venv sleep
simon@simon ~
-: kill 8981
[1]  + 8981 terminated  sleep 200
```

Process Control

killall - sends a signal to all processes running any of the specified commands.

```
 ✘ simon@simon  ~
-: sleep 1000 &
[2] 8817
 ⚙ simon@simon  ~
-: sleep 200 &
[3] 8835
 ⚙ simon@simon  ~
-: ps -ef | grep sleep
simon        8740    4363  0 14:55 pts/0    00:00:00 sleep 1000
simon        8817    4363  0 14:55 pts/0    00:00:00 sleep 1000
simon        8835    4363  0 14:55 pts/0    00:00:00 sleep 200
simon        8858    4363  0 14:55 pts/0    00:00:00 grep --color=auto --exclude-dir=.bzr
-exclude-dir=.tox --exclude-dir=.venv --exclude-dir=venv sleep
 ⚙ simon@simon  ~
-: killall sleep
[1]    8740 terminated  sleep 1000
[2]  - 8817 terminated  sleep 1000
[3]  + 8835 terminated  sleep 200
 simon@simon  ~
-: ps -ef | grep sleep
simon        8893    4363  0 14:56 pts/0    00:00:00 grep --color=auto --exclude-dir=.bzr
-exclude-dir=.tox --exclude-dir=.venv --exclude-dir=venv sleep
```

Process Control

killall - sends a signal to all processes running any of the specified commands.

```
 ✘ simon@simon  ~
-: sleep 1000 &
[2] 8817
 ⚙ simon@simon  ~
-: sleep 200 &
[3] 8835
 ⚙ simon@simon  ~
-: ps -ef | grep sleep
simon        8740    4363  0 14:55 pts/0    00:00:00 sleep 1000
simon        8817    4363  0 14:55 pts/0    00:00:00 sleep 1000
simon        8835    4363  0 14:55 pts/0    00:00:00 sleep 200
simon        8858    4363  0 14:55 pts/0    00:00:00 grep --color=auto --exclude-dir=.bzr
-exclude-dir=.tox --exclude-dir=.venv --exclude-dir=venv sleep
 ⚙ simon@simon  ~
-: killall sleep
[1]    8740 terminated  sleep 1000
[2]  - 8817 terminated  sleep 1000
[3]  + 8835 terminated  sleep 200
 simon@simon  ~
-: ps -ef | grep sleep
simon        8893    4363  0 14:56 pts/0    00:00:00 grep --color=auto --exclude-dir=.bzr
-exclude-dir=.tox --exclude-dir=.venv --exclude-dir=venv sleep
```

Jobs in Linux

A **job** in Linux refers to a process started from the shell that can be managed using job control commands. Jobs are typically foreground or background tasks initiated by the user.

Creating Jobs

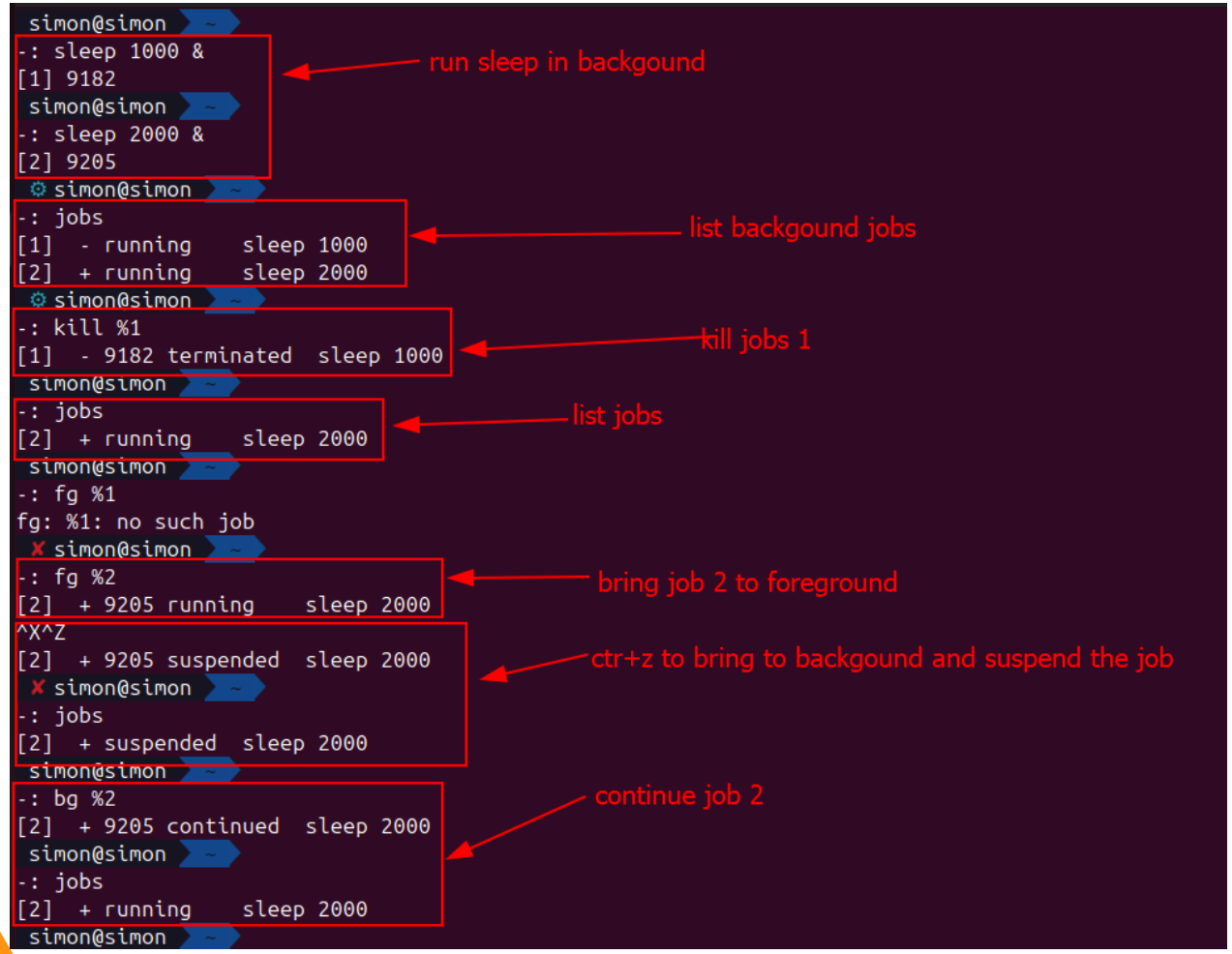
- You can create a job by running a command in the shell. To run it in the background, append an ampersand (&) to the command:

```
sleep 60 &
```

Managing Jobs

Command	Purpose
jobs	List current jobs in the shell
fg %1	Bring job 1 to the foreground
bg %2	Resume job 2 in the background
kill %1	Terminate job 1
Ctrl+Z	Pause a foreground job (sends it to background as stopped)

Jobs in Linux



Command	Purpose
jobs	List current jobs in the shell
fg %1	Bring job 1 to the foreground
bg %2	Resume job 2 in the background
kill %1	Terminate job 1
Ctrl+Z	Pause a foreground job (sends it to background as stopped)

Linux Service (Daemons)

A **daemon** is a background process that runs independently of user control. It typically starts when the system boots and continues running until shutdown.

Key Characteristics:

- **Runs in the background** without direct user interaction
- **Handles system tasks** like networking, printing, logging, scheduling, etc.
- **Often ends in `d`**, like ssh**d**, cron**d**, system**d**
- **Detached from terminal** and usually started by the system

Daemon	Purpose
sshd	Handles incoming SSH connections
crond	Executes scheduled tasks
systemd	Manages system startup and services
httpd	Web server daemon (Apache)

Service Management

systemctl is used to manage system services (daemons).

Command	Description
systemctl start nginx	Start a service
systemctl stop nginx	Stop a service
systemctl restart nginx	Restart a service
systemctl status nginx	Check service status
systemctl enable nginx	Enable service at boot
systemctl disable nginx	Disable service at boot

Package Management

VNTECHIES

Linux Package

What is Package

A **Linux package** is a bundled collection of files that make up a software application, including:

- The **program itself**
- Required **libraries**
- Configuration files
- Metadata (like version, dependencies, and description)

Think of it like a neatly packed box  that contains everything needed to install and run a piece of software on your system.

Why Packages Matter

Instead of manually downloading and placing files, packages simplify:

- **Installation**
- **Upgrades**
- **Removal**
- **Dependency resolution**

They're managed by **package managers** like apt, yum, dnf, or pacman, depending on your Linux distribution.

Package Management

Package Managers: apt (Advanced Package Tool)
Linux uses **package managers** to install, update, and manage software. On Ubuntu and Debian systems.

- Here are essential apt commands for managing packages:

Operation	Command Example	Description
Install	sudo apt install nginx	Installs a package
Update Index	sudo apt update	Refreshes package list
Upgrade	sudo apt upgrade	Updates installed packages
Remove	sudo apt remove nginx	Uninstalls a package
Search	apt search apache	Finds packages by name
List Installed	apt list --installed	Displays installed packages

Practice Lab